

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



THÍ NGHIỆM
XỬ LÝ TÍN HIỆU SỐ (CO2036)

BÀI THỰC HÀNH
LAB 3

HK252

Giảng viên hướng dẫn: Phạm Công Thái

STT	Họ và tên	MSSV	Lớp
1	Nguyễn Ngọc Hà My	2212104	L01

Thành phố Hồ Chí Minh, tháng 3 năm 2026



Mục lục

1	Giới thiệu	2
2	Mục tiêu	2
3	Phương pháp	2
4	Nội dung thực hành	3
4.1	Exercise 1: Delay	3
4.2	Exercise 2: Advance	6
4.3	Exercise 3: Fold	8
4.4	Exercise 4: Add	10
4.5	Exercise 5: Multiply	12
4.6	Exercise 6: Convolution	15
4.7	Bài 2.1	17
4.8	Bài 2.2	19
4.9	Bài 2.4	21
4.10	Bài 2.6	22
5	Bàn luận	26
	Nguồn mã nguồn	27
	Tài liệu tham khảo	27

Tổng Quan

Bài thực hành Lab 3 tập trung vào việc xử lý tín hiệu rời rạc bằng cách xây dựng các hàm thao tác cơ bản trong môi trường Scilab. Sinh viên được yêu cầu cài đặt các phép biến đổi tín hiệu như trễ, sớm, đảo, cộng, nhân và tích chập. Ngoài ra, bài thực hành còn giúp củng cố kiến thức về các phép biến đổi tín hiệu trong miền thời gian rời rạc thông qua các bài tập bổ sung.

1 Giới thiệu

Trong xử lý tín hiệu số, tín hiệu rời rạc được biểu diễn dưới dạng dãy số theo biến thời gian nguyên n . Các phép biến đổi như dịch thời gian, đảo tín hiệu, cộng và tích chập đóng vai trò quan trọng trong việc phân tích và thiết kế hệ thống.

Lab này giúp sinh viên hiểu rõ cách biểu diễn tín hiệu bằng vector, cách xác định vị trí gốc (origin), và cách thực hiện các phép toán trên tín hiệu bằng lập trình Scilab.

2 Mục tiêu

Sau khi hoàn thành bài thực hành, sinh viên có thể:

- Hiểu và biểu diễn tín hiệu rời rạc bằng vector trong Scilab.
- Thực hiện các phép biến đổi tín hiệu: delay, advance, fold.
- Thực hiện các phép toán giữa hai tín hiệu: cộng, nhân.
- Tính toán tích chập giữa hai tín hiệu.
- Vẽ đồ thị tín hiệu rời rạc và phân tích kết quả.
- Kiểm tra tính bất biến theo thời gian của hệ thống.

3 Phương pháp

Trong bài thực hành, phần mềm Scilab được sử dụng để mô phỏng và xử lý tín hiệu rời rạc.

Các bước thực hiện:

- Biểu diễn tín hiệu dưới dạng vector và xác định vị trí gốc.

- Xây dựng các hàm xử lý tín hiệu theo cú pháp:

```
function [output] = function_name(input)
    ...
endfunction
```

- Sử dụng các phép biến đổi chỉ số để thực hiện dịch tín hiệu.
- Sử dụng hàm `plot2d3` để vẽ tín hiệu rời rạc.
- Sử dụng hàm `conv()` để tính tích chập.
- Phân tích kết quả thông qua đồ thị và giá trị thu được.

4 Nội dung thực hành

4.1 Exercise 1: Delay

Trong bài này, thực hiện phép dịch tín hiệu theo thời gian (delay), được định nghĩa bởi:

$$y(n) = x(n - k)$$

với $k > 0$. Phép toán này làm dịch tín hiệu sang phải k mẫu, đồng thời vị trí gốc của tín hiệu thay đổi tương ứng.

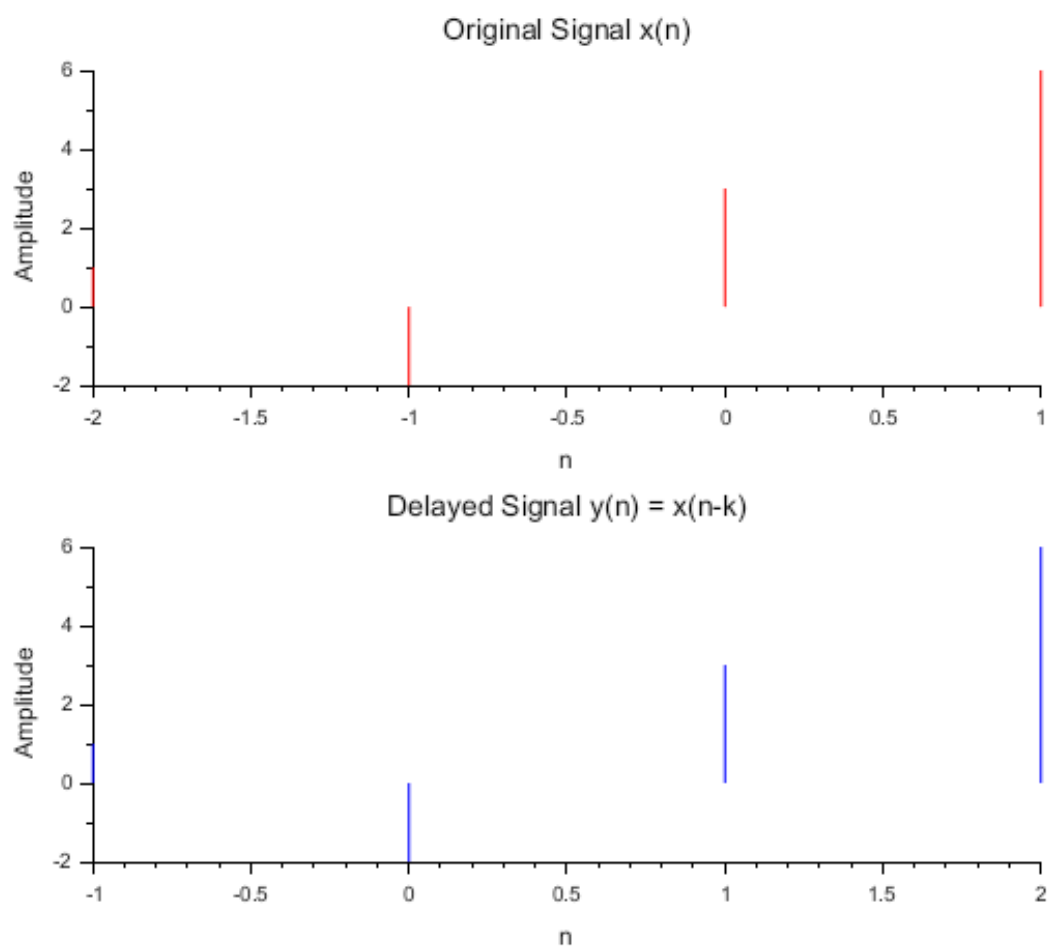
Tín hiệu đầu vào được biểu diễn dưới dạng vector và vị trí gốc được xác định bởi chỉ số `xorigin`. Sau khi thực hiện phép delay, tín hiệu đầu ra giữ nguyên giá trị nhưng trục thời gian bị dịch.

Kết quả thực hiện cho thấy tín hiệu đầu ra có cùng biên độ với tín hiệu ban đầu, nhưng vị trí các mẫu được dời sang phải.

```
1 clc;
2 clear;
3 clf;
4
5 function [yn, yorigin] = delay(xn, xorigin, k)
6
7     yorigin = xorigin - k;
8     yn = xn;
```

```
9
10 N = length(xn);
11 n_x = (1:N) - xorigin;
12 n_y = (1:N) - yorigin;
13
14 clf;
15 subplot(2,1,1);
16 plot2d3(n_x, xn, style=5);
17 xtitle("Original Signal x(n)", "n", "Amplitude");
18
19 subplot(2,1,2);
20 plot2d3(n_y, yn, style=2);
21 xtitle("Delayed Signal y(n) = x(n-k)", "n", "Amplitude");
22
23 endfunction
24
25 xn = [1, -2, 3, 6];
26 xorigin = 3;
27 k = 1;
28
29 [yn, yorigin] = delay(xn, xorigin, k);
30
31 disp("yn = ");
32 disp(yn);
33 disp("yorigin = ");
34 disp(yorigin);
```

Listing 1: Code exercise 1



Hình 1: Kết quả các phép biến đổi thời gian của tín hiệu rời rạc

4.2 Exercise 2: Advance

Phép advance được định nghĩa:

$$y(n) = x(n + k)$$

Khác với delay, phép advance làm dịch tín hiệu sang trái k mẫu. Về mặt cài đặt, tín hiệu đầu ra vẫn giữ nguyên giá trị nhưng chỉ số thời gian thay đổi.

Kết quả cho thấy các mẫu tín hiệu xuất hiện sớm hơn so với tín hiệu ban đầu.

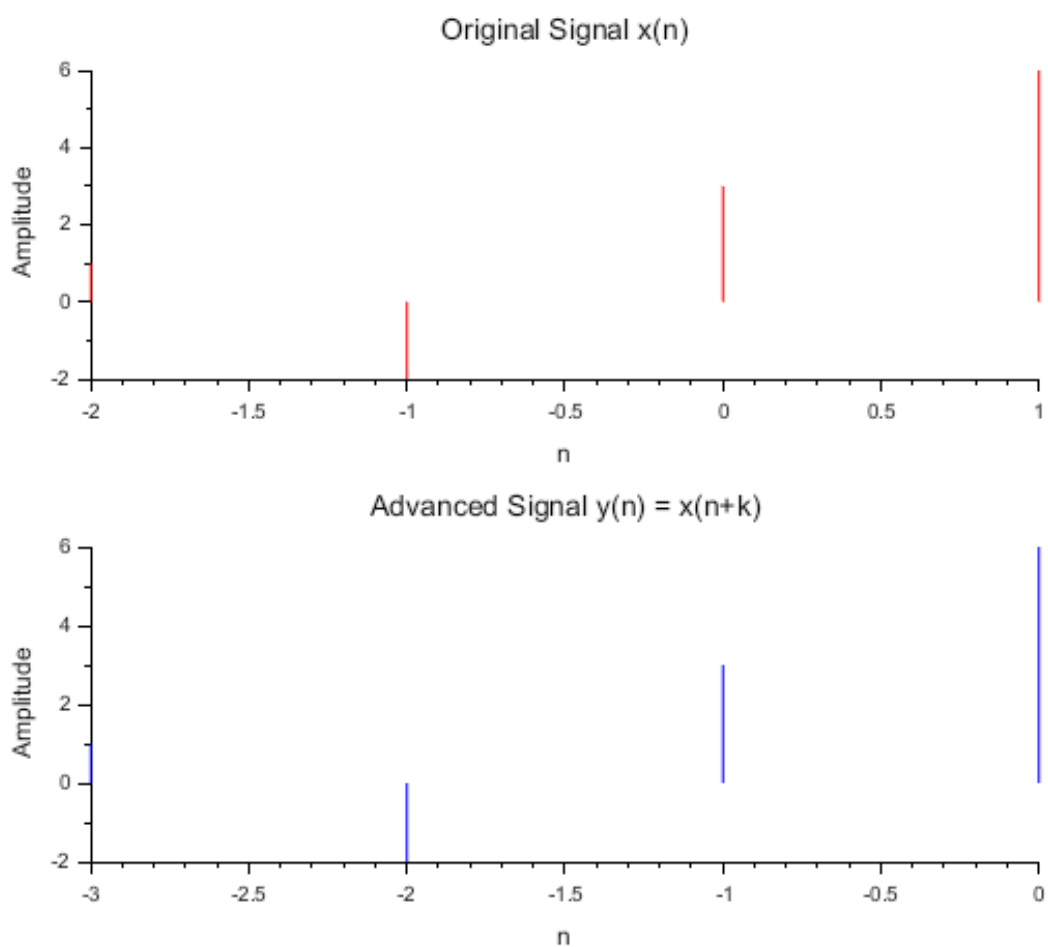
```
1 clc;
2 clear;
3 clf;
4
5 function [yn, yorigin] = advance(xn, xorigin, k)
6
7     yorigin = xorigin + k;
8     yn = xn;
9
10    N = length(xn);
11    n_x = (1:N) - xorigin;
12    n_y = (1:N) - yorigin;
13
14    clf;
15    subplot(2,1,1);
16    plot2d3(n_x, xn, style=5);
17    xtitle("Original Signal x(n)", "n", "Amplitude");
18
19    subplot(2,1,2);
20    plot2d3(n_y, yn, style=2);
21    xtitle("Advanced Signal y(n) = x(n+k)", "n", "Amplitude");
22
23 endfunction
24
25 xn = [1, -2, 3, 6];
26 xorigin = 3;
27 k = 1;
```

```

28
29 [yn, yorigin] = advance(xn, xorigin, k);
30
31 disp("yn = ");
32 disp(yn);
33 disp("yorigin = ");
34 disp(yorigin);

```

Listing 2: Code exercise 2



Hình 2: Kết quả các phép biến đổi thời gian của tín hiệu rời rạc

4.3 Exercise 3: Fold

Phép fold (đảo tín hiệu) được định nghĩa:

$$y(n) = x(-n)$$

Phép toán này đảo tín hiệu qua trục $n = 0$. Khi thực hiện, thứ tự các phần tử trong vector bị đảo ngược, đồng thời vị trí gốc cũng thay đổi.

Kết quả cho thấy tín hiệu đầu ra là phiên bản đối xứng của tín hiệu ban đầu qua trục thời gian.

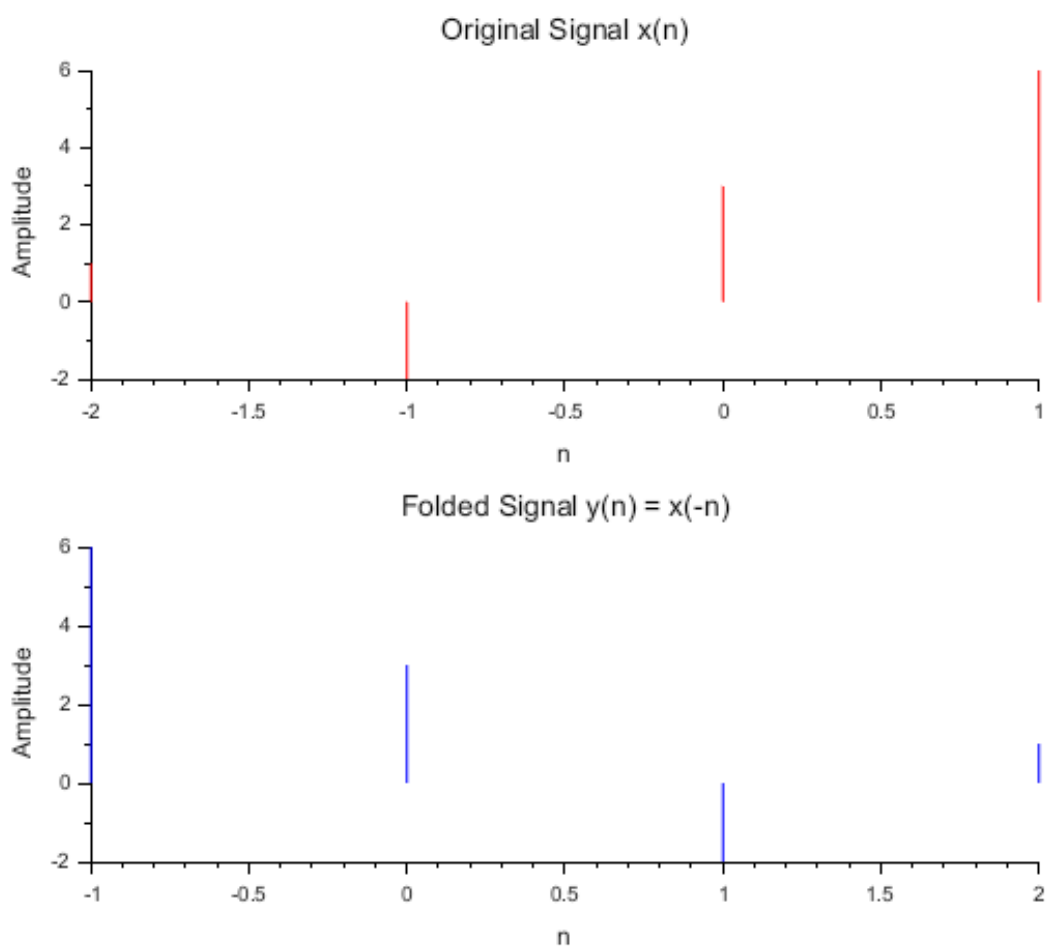
```
1 clc;
2 clear;
3 clf;
4
5 function [yn, yorigin] = fold(xn, xorigin)
6
7     yn = xn($:-1:1);
8     yorigin = length(xn) - xorigin + 1;
9
10    N = length(xn);
11    n_x = (1:N) - xorigin;
12    n_y = (1:N) - yorigin;
13
14    clf;
15    subplot(2,1,1);
16    plot2d3(n_x, xn, style=5);
17    xtitle("Original Signal x(n)", "n", "Amplitude");
18
19    subplot(2,1,2);
20    plot2d3(n_y, yn, style=2);
21    xtitle("Folded Signal y(n) = x(-n)", "n", "Amplitude");
22
23 endfunction
24
25 xn = [1, -2, 3, 6];
26 xorigin = 3;
```

```

27
28 [yn, yorigin] = fold(xn, xorigin);
29
30 disp("yn = ");
31 disp(yn);
32 disp("yorigin = ");
33 disp(yorigin);

```

Listing 3: Code exercise 3



Hình 3: Kết quả các phép biến đổi thời gian của tín hiệu rời rạc

4.4 Exercise 4: Add

Trong bài này, thực hiện phép cộng hai tín hiệu:

$$y(n) = x_1(n) + x_2(n)$$

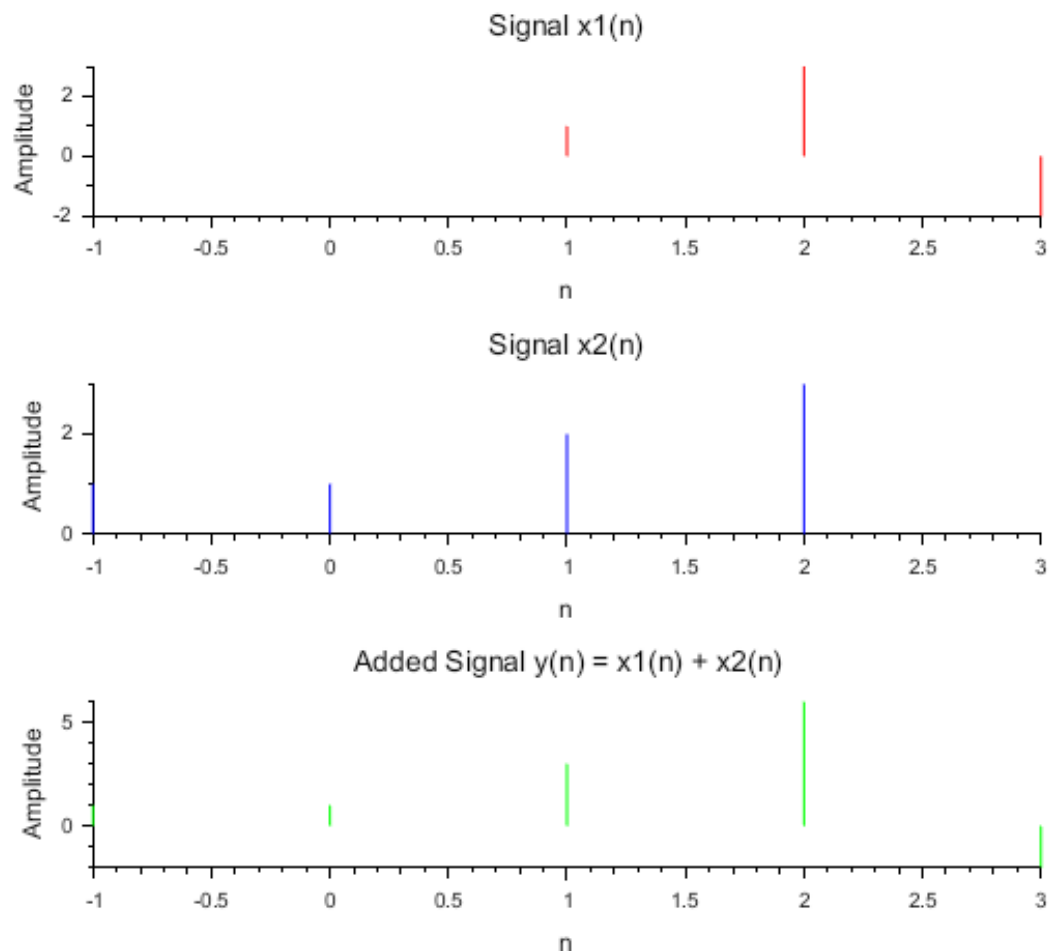
Do hai tín hiệu có thể có vị trí gốc khác nhau, cần thực hiện căn chỉnh trục thời gian trước khi cộng. Các giá trị ngoài miền xác định được xem là 0.

Kết quả cho thấy tín hiệu đầu ra là tổng của hai tín hiệu tại từng thời điểm tương ứng.

```
1 clc;
2 clear;
3 clf;
4
5 function [yn, yorigin] = add(x1n, x1origin, x2n, x2origin)
6
7     n1_start = 1 - x1origin;
8     n1_end   = length(x1n) - x1origin;
9
10    n2_start = 1 - x2origin;
11    n2_end   = length(x2n) - x2origin;
12
13    n_start = min(n1_start, n2_start);
14    n_end   = max(n1_end, n2_end);
15
16    n = n_start:n_end;
17    L = length(n);
18
19    x1a = zeros(1, L);
20    x2a = zeros(1, L);
21
22    for i = 1:L
23        idx1 = n(i) + x1origin;
24        if idx1 >= 1 & idx1 <= length(x1n) then
25            x1a(i) = x1n(idx1);
26        end
```

```
27     idx2 = n(i) + x2origin;
28     if idx2 >= 1 & idx2 <= length(x2n) then
29         x2a(i) = x2n(idx2);
30     end
31 end
32
33 yn = x1a + x2a;
34 yorigin = 1 - n_start;
35
36 clf;
37 subplot(3,1,1);
38 plot2d3(n, x1a, style=5);
39 xtitle("Signal x1(n)", "n", "Amplitude");
40
41 subplot(3,1,2);
42 plot2d3(n, x2a, style=2);
43 xtitle("Signal x2(n)", "n", "Amplitude");
44
45 subplot(3,1,3);
46 plot2d3(n, yn, style=3);
47 xtitle("Added Signal y(n) = x1(n) + x2(n)", "n", "Amplitude");
48
49 endfunction
50
51 x1n = [0, 1, 3, -2];
52 x1origin = 1;
53 x2n = [1, 1, 2, 3];
54 x2origin = 2;
55
56 [yn, yorigin] = add(x1n, x1origin, x2n, x2origin);
57
58 disp("yn = ");
59 disp(yn);
60 disp("yorigin = ");
61 disp(yorigin);
```

Listing 4: Code exercise 4



Hình 4: Kết quả các phép biến đổi thời gian của tín hiệu rời rạc

4.5 Exercise 5: Multiply

Phép nhân hai tín hiệu được định nghĩa:

$$y(n) = x_1(n) \cdot x_2(n)$$

Tương tự phép cộng, cần căn chỉnh hai tín hiệu về cùng trục thời gian. Sau đó thực hiện phép nhân từng phần tử.

Kết quả cho thấy tín hiệu đầu ra phản ánh sự tương tác giữa hai tín hiệu đầu vào.

```
1 clc;
2 clear;
3 clf;
4
5 function [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin)
```

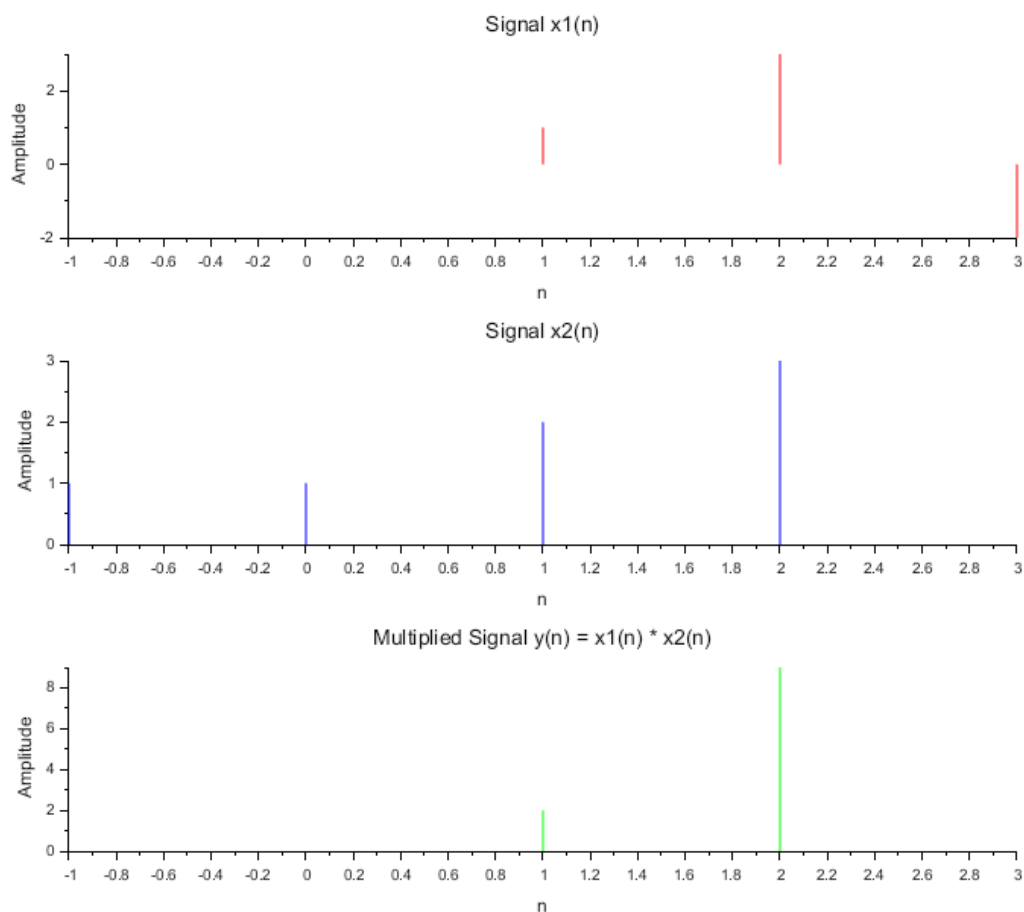
```
6     n1_start = 1 - x1origin;
7     n1_end   = length(x1n) - x1origin;
8     n2_start = 1 - x2origin;
9     n2_end   = length(x2n) - x2origin;
10    n_start = min(n1_start, n2_start);
11    n_end   = max(n1_end, n2_end);
12    n = n_start:n_end;
13    L = length(n);
14    x1a = zeros(1, L);
15    x2a = zeros(1, L);
16
17    for i = 1:L
18        idx1 = n(i) + x1origin;
19        if idx1 >= 1 & idx1 <= length(x1n) then
20            x1a(i) = x1n(idx1);
21        end
22        idx2 = n(i) + x2origin;
23        if idx2 >= 1 & idx2 <= length(x2n) then
24            x2a(i) = x2n(idx2);
25        end
26    end
27
28    yn = x1a .* x2a;
29    yorigin = 1 - n_start;
30    clf;
31    subplot(3,1,1);
32    plot2d3(n, x1a, style=5);
33    xtitle("Signal x1(n)", "n", "Amplitude");
34    subplot(3,1,2);
35    plot2d3(n, x2a, style=2);
36    xtitle("Signal x2(n)", "n", "Amplitude");
37    subplot(3,1,3);
38    plot2d3(n, yn, style=3);
39    xtitle("Multiplied Signal y(n) = x1(n) * x2(n)", "n", "Amplitude"
40    );
41 endfunction
```

```

41
42 x1n = [0, 1, 3, -2];
43 x1origin = 1;
44 x2n = [1, 1, 2, 3];
45 x2origin = 2;
46 [yn, yorigin] = multi(x1n, x1origin, x2n, x2origin);
47 disp("yn = ");
48 disp(yn);
49 disp("yorigin = ");
50 disp(yorigin);

```

Listing 5: Code exercise 5



Hình 5: Kết quả các phép biến đổi thời gian của tín hiệu rời rạc

4.6 Exercise 6: Convolution

Tích chập giữa hai tín hiệu được định nghĩa:

$$y(n) = x(n) * h(n)$$

Đây là phép toán quan trọng trong hệ LTI, biểu diễn đầu ra của hệ khi biết tín hiệu vào và đáp ứng xung.

Trong Scilab, phép tích chập được thực hiện bằng hàm `conv()`.

Kết quả cho thấy tín hiệu đầu ra có chiều dài lớn hơn và phụ thuộc vào cả hai tín hiệu đầu vào.

```
1 clc;
2 clear;
3 clf;
4
5 function [yn, yorigin] = convolution(xn, xorigin, hn, horigin)
6     yn = conv(xn, hn);
7     yorigin = xorigin + horigin - 1;
8     Nx = length(xn);
9     Nh = length(hn);
10    Ny = length(yn);
11    n_x = (1:Nx) - xorigin;
12    n_h = (1:Nh) - horigin;
13    n_y = (1:Ny) - yorigin;
14    clf;
15    subplot(3,1,1);
16    plot2d3(n_x, xn, style=5);
17    xtitle("Input Signal x(n)", "n", "Amplitude");
18
19    subplot(3,1,2);
20    plot2d3(n_h, hn, style=2);
21    xtitle("Impulse Response h(n)", "n", "Amplitude");
22
23    subplot(3,1,3);
24    plot2d3(n_y, yn, style=3);
```

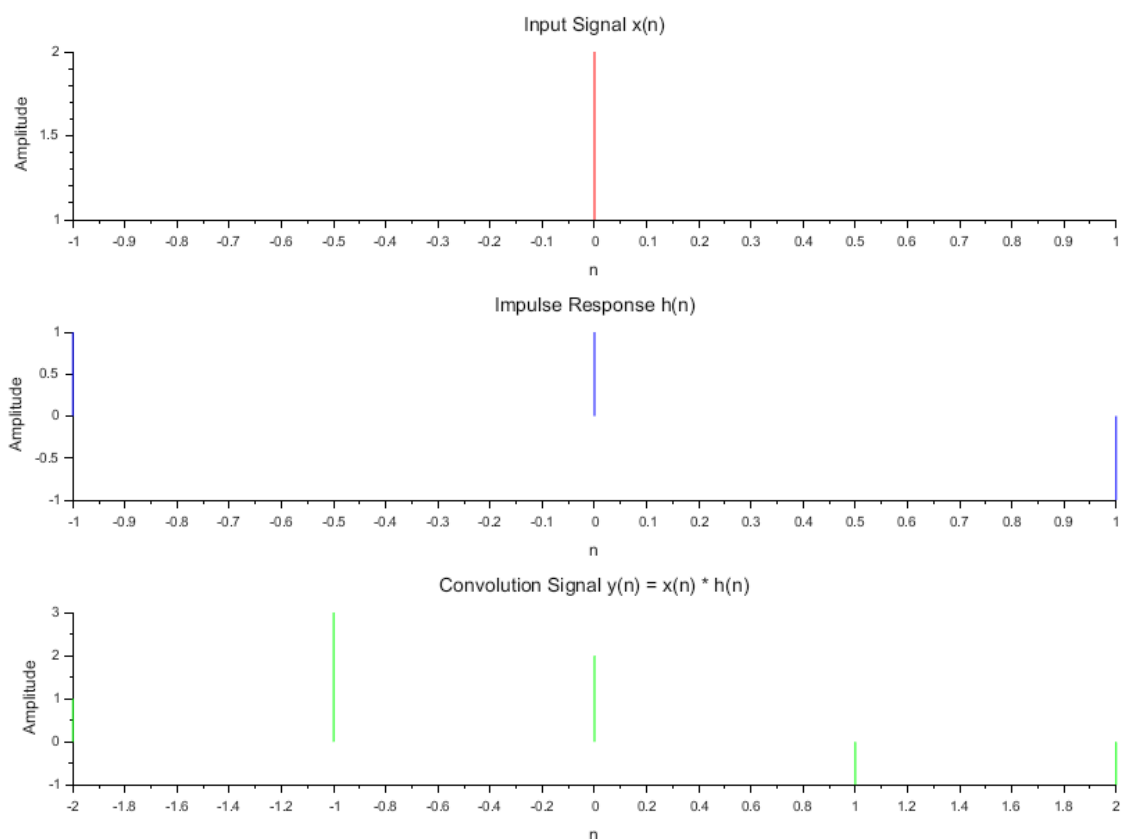


```

25     xtitle("Convolution Signal  $y(n) = x(n) * h(n)$ ", "n", "Amplitude")
26     ;
27 endfunction
28
29 xn = [1, 2, 1];
30 xorigin = 2;
31 hn = [1, 1, -1];
32 horigin = 2;
33 [yn, yorigin] = convolution(xn, xorigin, hn, horigin);
34 disp("yn = ");
35 disp(yn);
36 disp("yorigin = ");
37 disp(yorigin);

```

Listing 6: Code exercise 6



Hình 6: Kết quả các phép biến đổi thời gian của tín hiệu rời rạc

4.7 Bài 2.1

Bài toán: Cho tín hiệu:

$$x(n) = \begin{cases} 1 + \frac{n}{3}, & -3 \leq n \leq -1 \\ 1, & 0 \leq n \leq 3 \\ 0, & \text{otherwise} \end{cases}$$

(a) Xác định và vẽ $x(n)$

Tính các giá trị:

$$x(n) = \left\{ 0, \frac{1}{3}, \frac{2}{3}, 1^\uparrow, 1, 1, 1 \right\}$$

Tín hiệu được khai báo dưới dạng vector trong Scilab và vẽ bằng `plot2d3()` để kiểm tra kết quả.

(b) Thực hiện các phép biến đổi

1. Fold rồi delay 4 mẫu

Fold:

$$x(-n) = \left\{ 1, 1, 1, 1^\uparrow, \frac{2}{3}, \frac{1}{3}, 0 \right\}$$

Delay:

$$x(-n+4) = \left\{ 1, 1, 1, 1, \frac{2}{3}^\uparrow, \frac{1}{3}, 0 \right\}$$

2. Delay rồi fold

Delay:

$$x(n-4) = \left\{ 0, \frac{1}{3}, \frac{2}{3}, 1, 1^\uparrow, 1, 1 \right\}$$

Fold:

$$x(-n-4) = \left\{ 1, 1, 1, 1^\uparrow, \frac{2}{3}, \frac{1}{3}, 0 \right\}$$

(c) Vẽ $x(-n+4)$

Tín hiệu:

$$x(-n+4) = \left\{ 1, 1, 1, 1, \frac{2}{3}^\uparrow, \frac{1}{3}, 0 \right\}$$

(d) Nhận xét

Hai kết quả khác nhau:

$$x(-n+4) \neq x(-n-4)$$

Thấy được phép fold và delay không giao hoán.

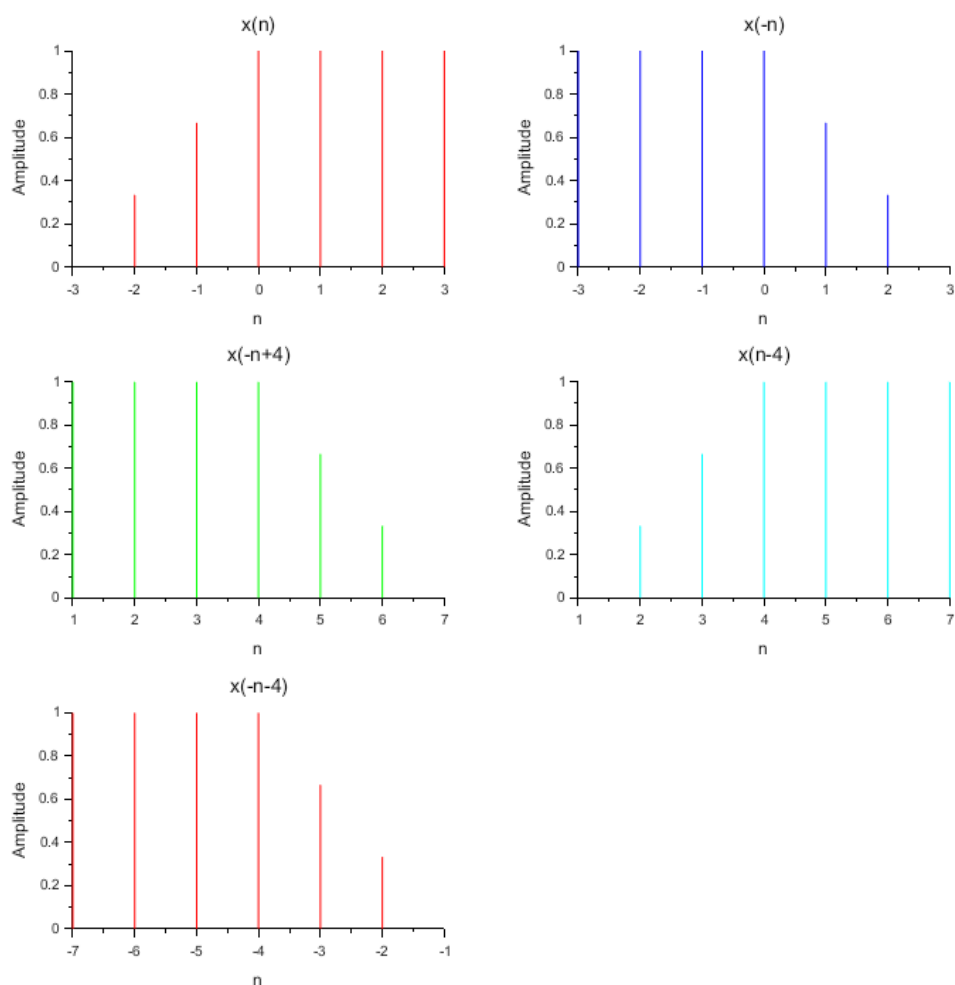
(e) Biểu diễn tín hiệu

$$x(n] = \frac{1}{3}\delta(n+2) + \frac{2}{3}\delta(n+1) + \delta(n) + \delta(n-1) + \delta(n-2) + \delta(n-3)$$

Hoặc:

$$x(n) = \left(1 + \frac{n}{3}\right) [u(n+3) - u(n)] + [u(n) - u(n-4)]$$

Hình vẽ thể hiện các phép biến đổi của tín hiệu $x(n)$: (a) tín hiệu gốc $x(n)$, (b) tín hiệu đảo $x(-n)$, (c) $x(-n+4)$ (fold rồi delay), (d) $x(n-4)$ (delay), (e) $x(-n-4)$ (delay rồi fold).



Hình 7: Các phép biến đổi của tín hiệu $x(n)$

4.8 Bài 2.2

Bài toán: Cho tín hiệu $x(n)$ từ hình P2.2:

$$x(n) = \begin{cases} 1, & -1 \leq n \leq 2 \\ \frac{1}{2}, & n = 3, 4 \\ 0, & \text{otherwise} \end{cases}$$

Suy ra:

$$x(n) = \left\{ 1, 1^\uparrow, 1, 1, \frac{1}{2}, \frac{1}{2} \right\}$$

Tín hiệu được khai báo trong Scilab dưới dạng vector và thực hiện các phép biến đổi theo yêu cầu.

Các phép biến đổi:

- $x(n-2)$:

$$x(n-2) = \left\{ 1, 1, 1^\uparrow, 1, \frac{1}{2}, \frac{1}{2} \right\}$$

- $x(4-n)$:

$$x(4-n) = \left\{ \frac{1}{2}, \frac{1}{2}, 1, 1, 1^\uparrow, 1 \right\}$$

- $x(n+2)$:

$$x(n+2) = \left\{ 1^\uparrow, 1, \frac{1}{2}, \frac{1}{2} \right\}$$

- $x(n)u(2-n)$:

$$x(n)u(2-n) = \left\{ 1, 1^\uparrow, 1, 1, 0, 0 \right\}$$

- $x(n-1)\delta(n-3)$:

$$x(n-1)\delta(n-3) = \left\{ 0^\uparrow, 0, 0, 1, 0, 0 \right\}$$

- $x(n^2)$:

$$x(n^2) = \left\{ \frac{1}{2}, 1, 1^\uparrow, 1, \frac{1}{2} \right\}$$

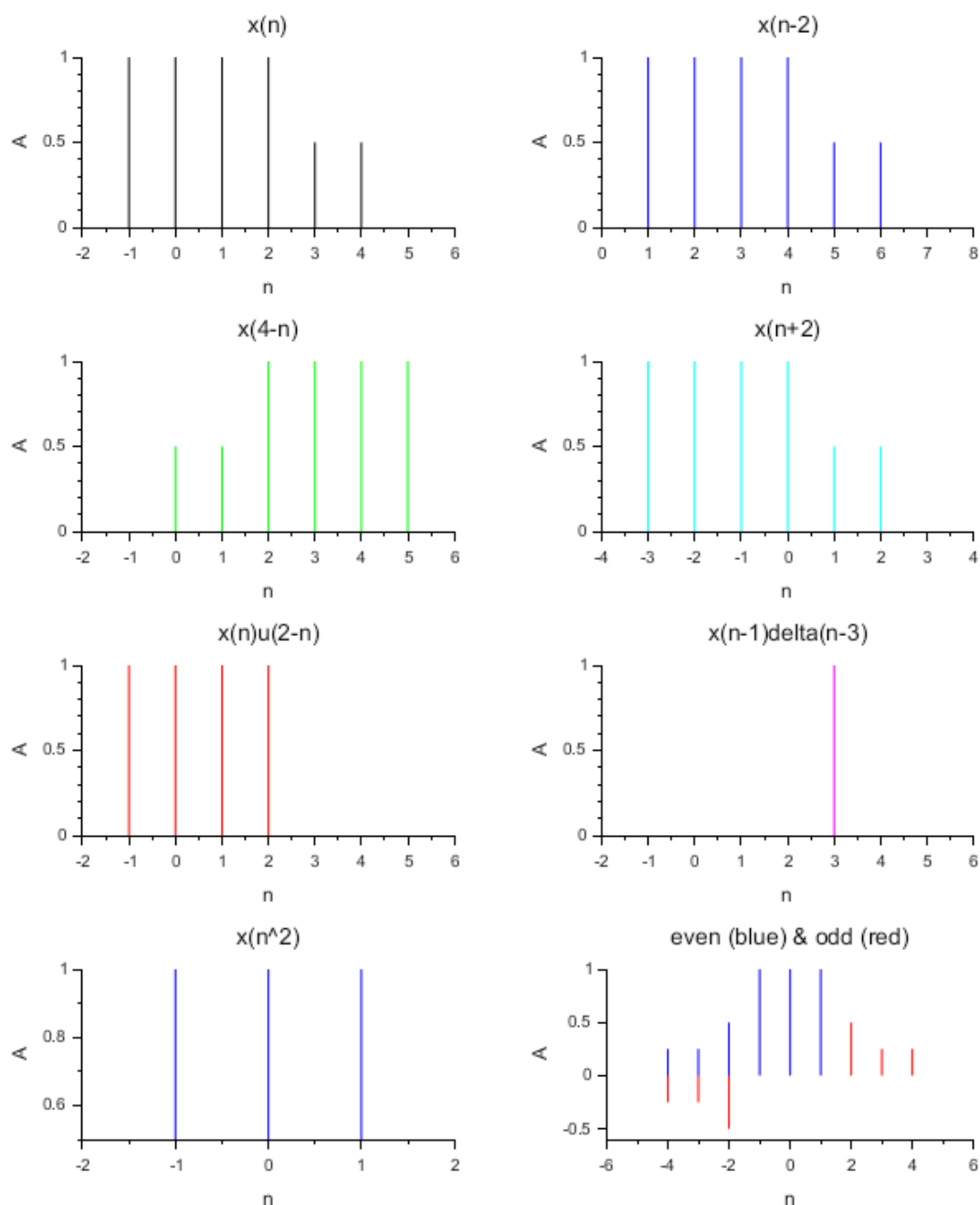
- Thành phần chẵn:

$$x_e(n) = \frac{x(n) + x(-n)}{2}$$

- Thành phần lẻ:

$$x_o(n) = \frac{x(n) - x(-n)}{2}$$

Các tín hiệu trên được vẽ trong Scilab dưới dạng các subplot để quan sát trực tiếp sự thay đổi theo từng phép biến đổi.



Hình 8: Các phép biến đổi của tín hiệu $x(n]$

Nhận xét: Các phép dịch và đảo làm thay đổi vị trí của tín hiệu, trong khi các

phép nhân với $u(n)$ và $\delta(n)$ giúp giới hạn hoặc chọn mẫu tín hiệu. Phân tích chẵn/lẻ giúp nhận biết tính đối xứng của tín hiệu.

4.9 Bài 2.4

Bài toán: Cho tín hiệu:

$$x(n) = \{2, 3, 4^\uparrow, 5, 6\}$$

Tín hiệu được khai báo trong Scilab với vị trí gốc tại phần tử có giá trị 4.

Phân tích thành phần chẵn và lẻ

Ta có:

$$x(-n) = \{6, 5, 4^\uparrow, 3, 2\}$$

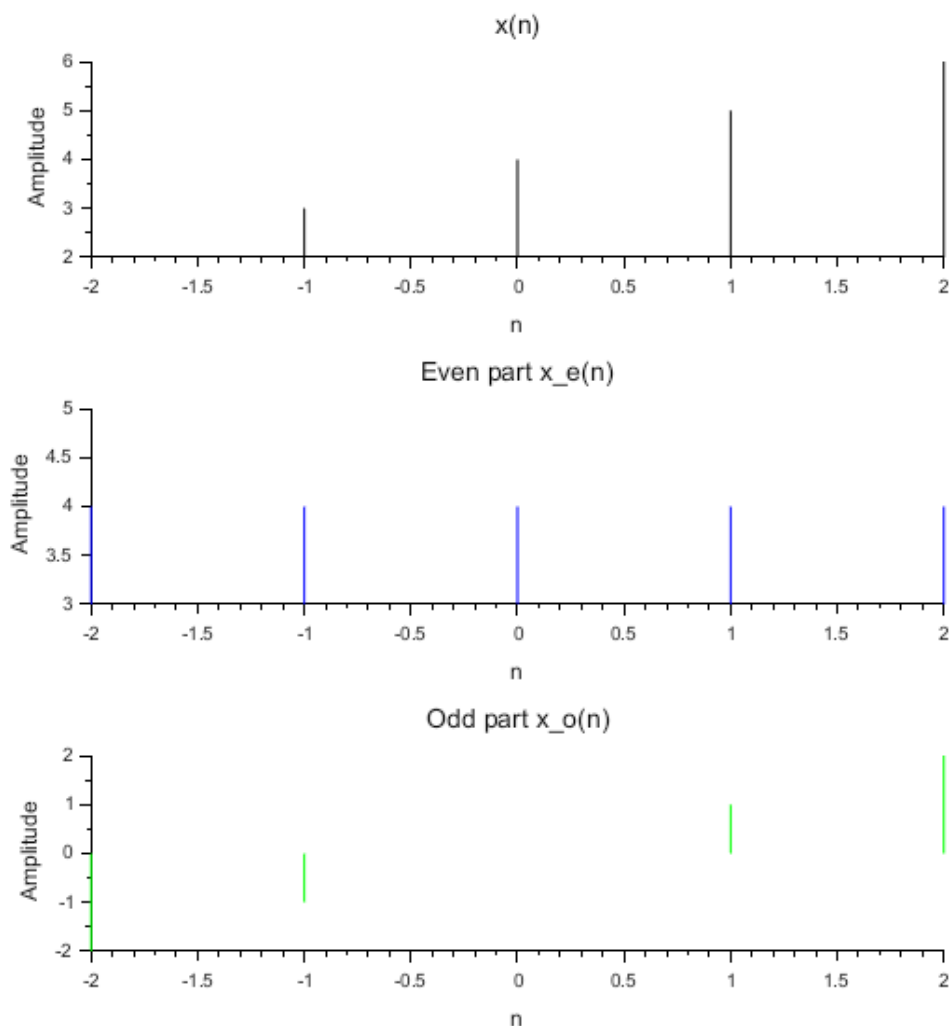
$$x_e(n) = \frac{x(n) + x(-n)}{2} \Rightarrow x_e(n) = \{4, 4, 4^\uparrow, 4, 4\}$$

$$x_o(n) = \frac{x(n) - x(-n)}{2} \Rightarrow x_o(n) = \{-2, -1, 0^\uparrow, 1, 2\}$$

Kiểm tra:

$$x(n) = x_e(n) + x_o(n)$$

Các tín hiệu được vẽ trong Scilab dưới dạng subplot gồm tín hiệu gốc, phần chẵn và phần lẻ.



Hình 9: Tín hiệu $x(n)$ và các thành phần chẵn $x_e(n)$, lẻ $x_o(n)$ trong Scilab.

Nhận xét: Mọi tín hiệu rời rạc đều có thể phân tích thành tổng của phần chẵn và phần lẻ, và phép phân tích này là duy nhất.

4.10 Bài 2.6

Bài toán: Kiểm tra tính bất biến theo thời gian của các hệ.

Tín hiệu vào:

$$x(n) = \begin{cases} 1, & 0 \leq n \leq 3 \\ 0, & \text{otherwise} \end{cases} \Rightarrow x(n) = \{1^\uparrow, 1, 1, 1\}$$

Nguyên tắc kiểm tra:

So sánh:

$$y(n-2) \quad \text{và} \quad y_2(n) = \mathcal{T}[x(n-2)]$$

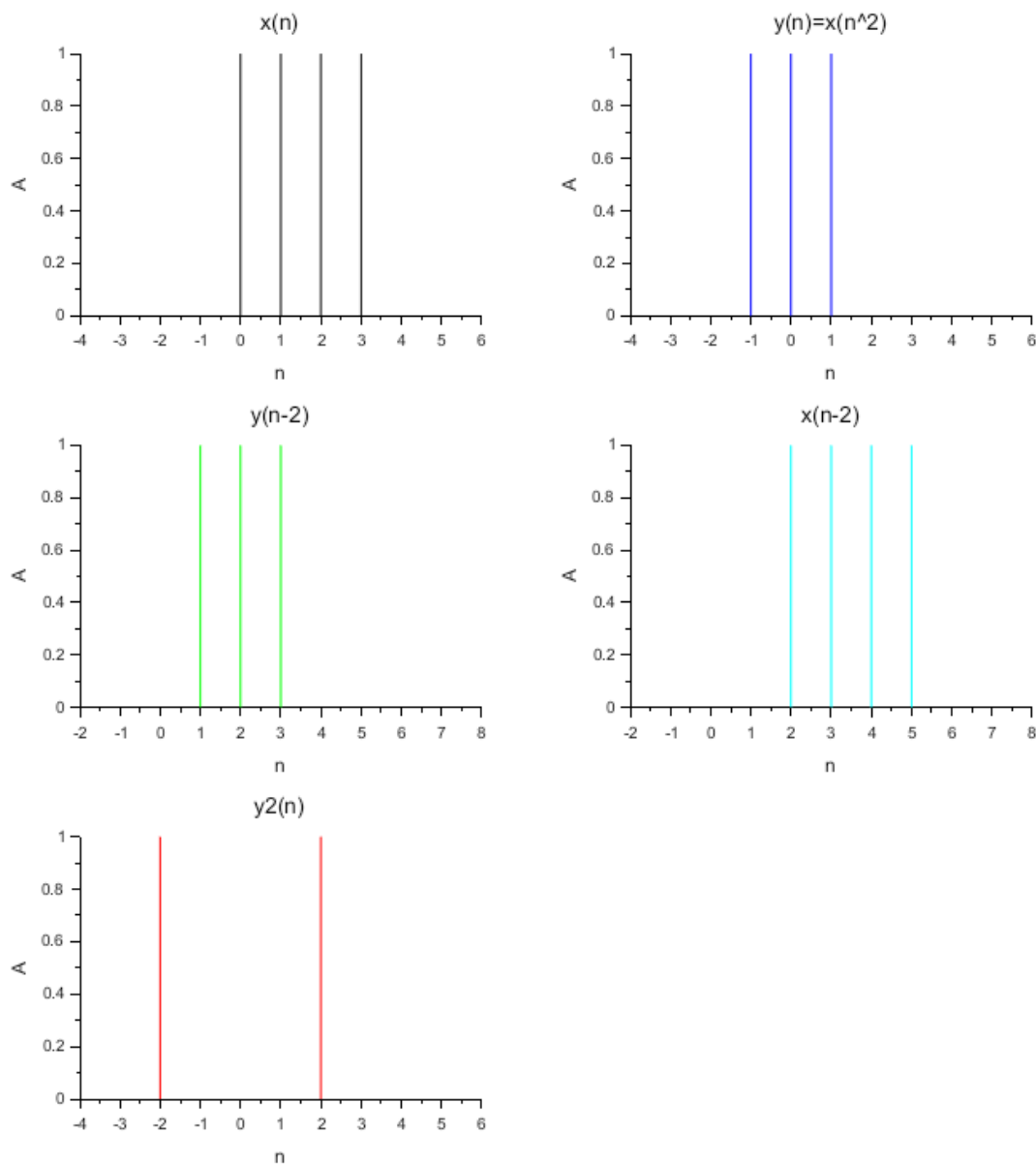
Nếu:

$$y_2(n) = y(n-2) \Rightarrow \text{hệ bất biến theo thời gian}$$

(a) Hệ $y(n] = x(n^2)$

$$y_2(n) \neq y(n-2)$$

$\Rightarrow$ Hệ không bất biến theo thời gian

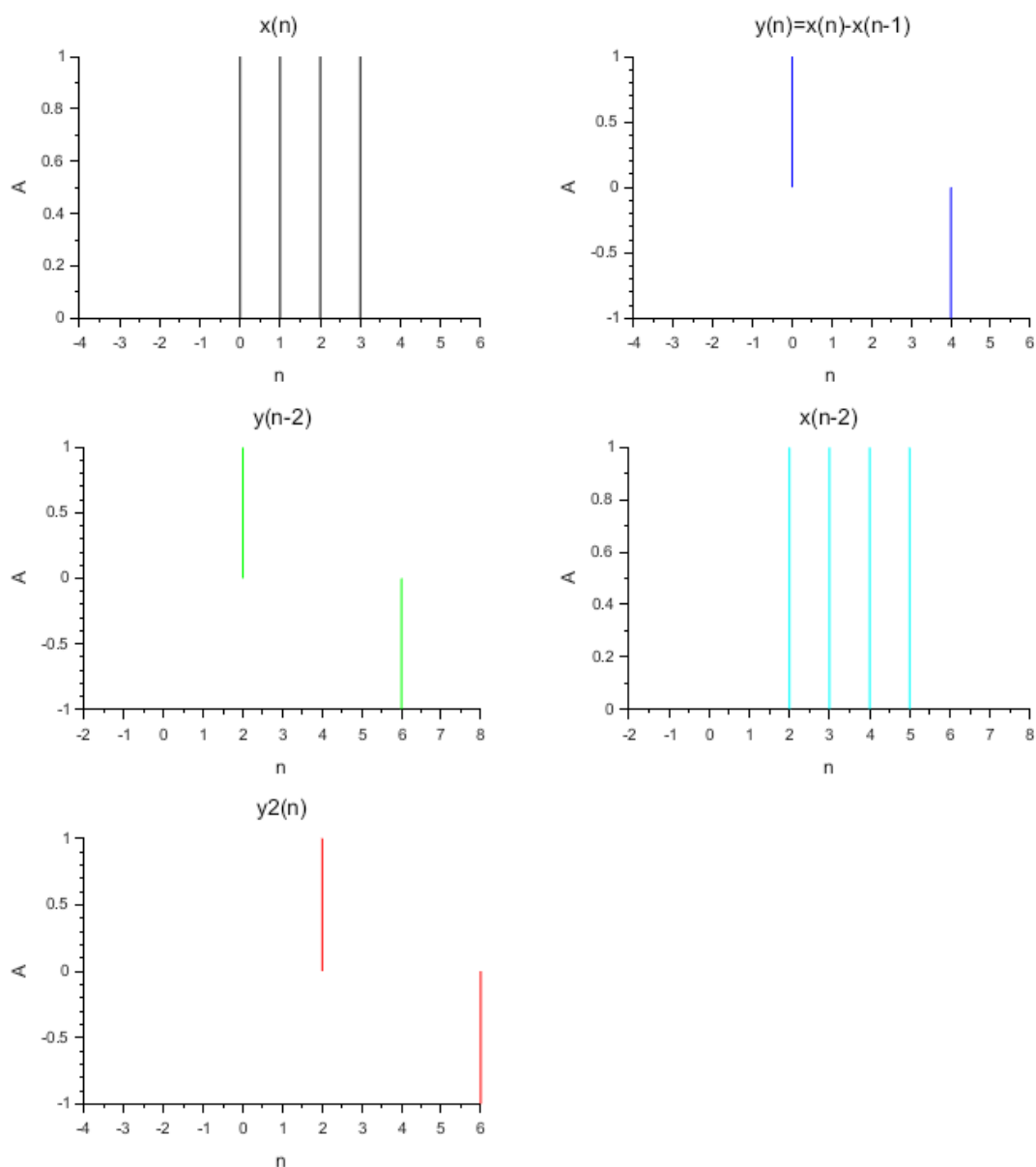


Hình 10: Kết quả kiểm tra hệ $y(n] = x(n^2)$ trong Scilab.

(b) Hệ $y(n] = x(n) - x(n - 1)$

$$y_2(n) = y(n - 2)$$

=> Hệ bất biến theo thời gian

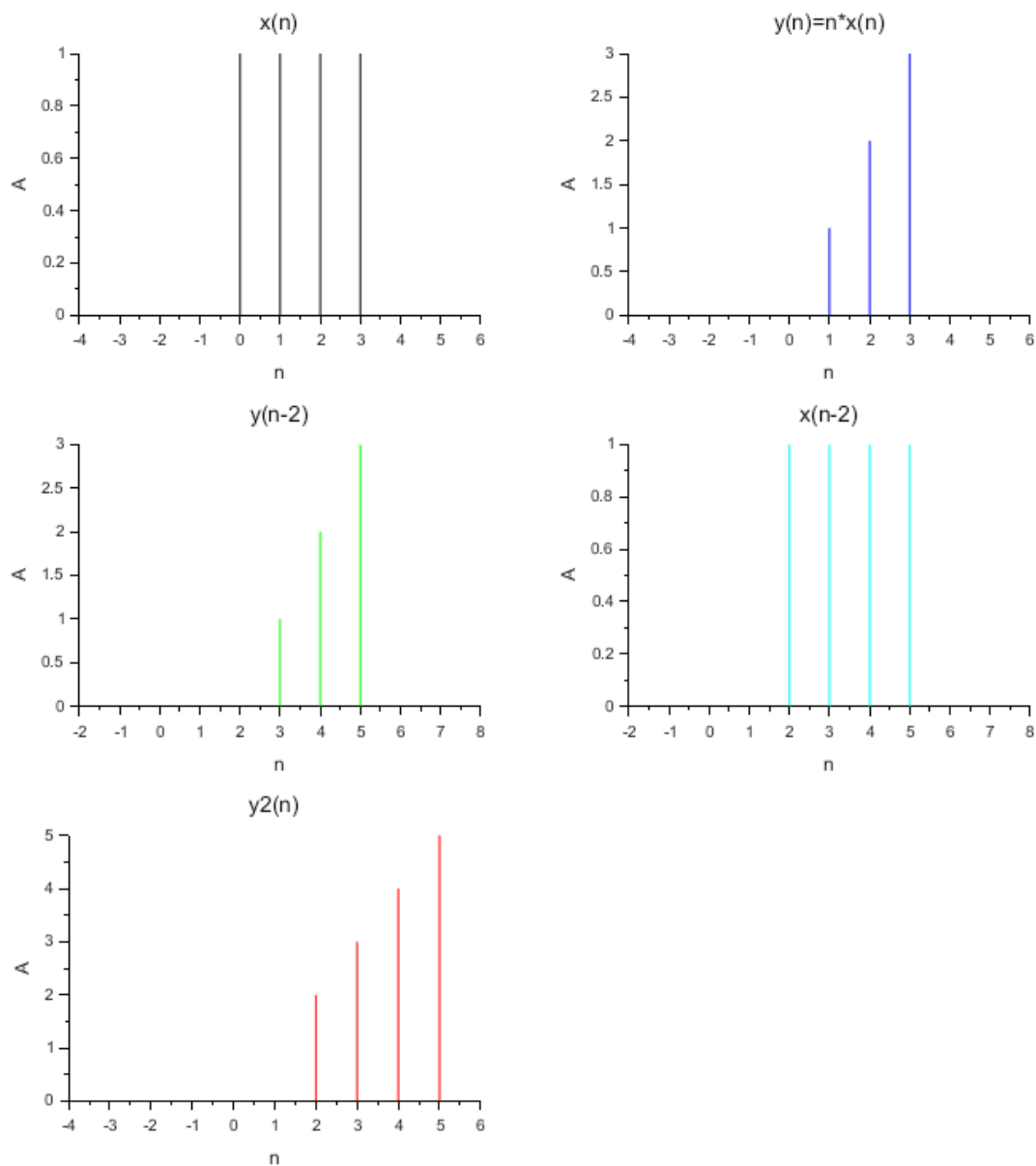


Hình 11: Kết quả kiểm tra hệ $y(n] = x(n) - x(n - 1]$ trong Scilab.

(c) Hệ $y(n] = n x(n]$

$$y_2(n) \neq y(n - 2)$$

=> Hệ không bất biến theo thời gian



Hình 12: Kết quả kiểm tra hệ $y(n) = n x(n)$ trong Scilab.

Nhận xét: Các hệ phụ thuộc trực tiếp vào biến n hoặc biến đổi phi tuyến như $x(n^2)$ thường không bất biến theo thời gian, trong khi hệ sai phân tuyến tính như $x(n) - x(n - 1)$ là bất biến.

5 Bàn luận

Qua bài thực hành, các phép biến đổi tín hiệu rời rạc từ cơ bản đến nâng cao đã được thực hiện bằng Scilab. Ở các bài Ex1–Ex3, các phép dịch thời gian (delay, advance) và phép đảo (fold) cho thấy tín hiệu chỉ thay đổi vị trí hoặc đối xứng theo trục thời gian mà không làm thay đổi biên độ.

Trong các bài Ex4–Ex6, các phép toán trên tín hiệu như cộng, nhân và tích chập được thực hiện, trong đó tích chập thể hiện rõ mối quan hệ giữa tín hiệu vào và đáp ứng của hệ LTI.

Ở phần bài tập bổ sung, bài 2.1 cho thấy thứ tự thực hiện các phép biến đổi là quan trọng, cụ thể fold và delay không giao hoán. Bài 2.2 minh họa các phép biến đổi phức tạp hơn như thay đổi đối số, nhân với $u(n)$ và $\delta(n)$, cũng như phân tích thành phần chẵn và lẻ của tín hiệu. Bài 2.4 khẳng định mọi tín hiệu đều có thể tách thành phần chẵn và lẻ. Cuối cùng, bài 2.6 giúp kiểm tra tính bất biến theo thời gian của hệ, trong đó hệ sai phân tuyến tính là bất biến, còn các hệ phụ thuộc vào n hoặc phi tuyến thì không.

Nhìn chung, việc sử dụng Scilab giúp trực quan hóa các phép biến đổi và kiểm chứng các kết quả lý thuyết một cách rõ ràng.



Nguồn mã nguồn

- Link Github: https://github.com/mimin514/LAB_DSP

Tài liệu tham khảo

Tài liệu

- [1] Department of Electrical Engineering, HCMUT, *Digital Signal Processing Laboratory Manual*, Ho Chi Minh City University of Technology.
- [2] Scilab Enterprises, *Scilab: Free and Open Source Software for Numerical Computation*. Available: <https://www.scilab.org/>
- [3] Scilab Enterprises, *Scilab Beginner's Guide*. Available: https://www.scilab.org/sites/default/files/Scilab_beginners.pdf